



Statement-Level Control Structures

CS3204N Programming Languages

Flow of Control in Programs

A program executes statements in a certain **order** called the **execution sequence**.

Three levels of control flow:

- Within expressions
- Among statements
- Among program units

Control Statements

A statement that determines the **order in which other statements are executed** in a program.

Purpose:

- Control program flow
- Enable decisions
- Enable repetition

Control Structures

A combination of a **control statement** and the **statements whose execution it controls**.

Example:

`if(x > y)` → control statement

`max = x;` → controlled statement

Importance of Control Structures

Without control structures:

- Programs would execute **only sequentially**
- Complex algorithms could not be implemented

Two fundamental programming needs:

- **Selection**
- **Iteration**

Selection

A control mechanism that allows a program to **choose between two or more alternative paths of execution** based on a condition.

Example:

```
if (score >= 75)  
    print("Passed");
```

Selection Statements

A control statement that allows the program to **select one of several possible execution paths**.

Common examples:

- if
- if-else
- switch
- case

Two-Way Selection Statement

A selection statement that allows the program to choose **between two alternative paths**.

General form:

if (condition)

 statement1

else

 statement2

Control Expression

An expression in a selection or iteration statement that is **evaluated to determine which statements should be executed.**

Example: **if** ($x > 10$)

Here: $x > 10 \rightarrow$ control expression

Then Clause and Else Clause

Then Clause

The part of an if statement executed when the **control expression evaluates to true**.

Else Clause

The part executed when the **control expression evaluates to false**.

Then Clause and Else Clause

if ($x > y$)

$\text{max} = x;$ $\leftarrow$ then clause

else

$\text{max} = y;$ $\leftarrow$ else clause

Nested Selector

A selection statement that appears **inside another selection statement**.

Example:

```
if(x > 0)
    if(y > 0)
        print("positive")
```

Dangling Else Problem

An ambiguity that occurs in nested if statements when it is **unclear which if statement an else clause belongs to.**

Example:

```
if (x > 0)
    if (y > 0)
        print("A")
    else
        print("B")
```

Question:

Which if does the else belong to?

Solutions to the Dangling Else Problem

Programming languages solve the dangling else problem by:

- Associating the **else** with the **nearest unmatched if**
- Using **block delimiters**
- Using **explicit terminators**

Solutions to the Dangling Else Problem

Language	Solution
C / Java	{ }
Ada	end if
Python	indentation

Multiple Selection

A selection statement that allows the program to **choose among several alternative execution paths**.

```
switch(day)
{
    case 1: Monday
    case 2: Tuesday
}
```


Selector Expression

The expression whose value determines **which case branch will be executed in a multiple selection statement.**

switch(day)

day → selector expression

Case Label

A constant value associated with a branch in a multiple selection statement that **determines when that branch is executed**.

Example:

case 1:

Default Clause

An optional branch executed when **none of the case labels match the selector expression.**

Example:

default:

```
print("Invalid input")
```

Iterative Statements

A control statement that **repeatedly executes a block of code as long as a specified condition holds.**

Examples

- for
- while
- do-while

Iteration

The repeated execution of a block of code **within a loop structure**.

Example:

- Printing numbers 1–10.

Counter-Controlled Loop

A loop whose number of iterations is controlled by a **loop variable that changes by a fixed amount each iteration.**

Example:

```
for(i = 1; i <= 10; i++)
```

Loop Variable

A variable that controls the **number of times a loop executes**.

Example

```
for(i = 0; i < 10; i++)
```

i → loop variable

Stepsize

The amount by which the loop variable changes during each iteration.

Example

$$i = i + 2$$

$$\text{Stepsize} = 2$$

Logically Controlled Loop

A loop whose execution depends on the **truth value of a Boolean expression**.

Examples

- while loop
- do-while loop

Pretest Loop

A loop in which the **control expression is evaluated before the loop body executes.**

Example

- `while(condition)`
- If condition is false initially $\rightarrow$ loop never executes.

Posttest Loop

A loop in which the **control expression is evaluated after the loop body executes.**

Example

```
do  
{  
}  
while(condition)
```

Loop executes **at least once.**

User-Located Loop Control Mechanism

A loop control method where the programmer **places statements inside the loop body to alter loop execution.**

Examples

- break
- continue

Iteration Based on Data Structures

A loop mechanism that **automatically iterates over elements of a collection or data structure.**

Example

- Python
- for item in list

Iterator

An object or mechanism that allows a program to **traverse the elements of a data structure sequentially**.

Example

- Iterating through a list or array.

Unconditional Branch

A control statement that **transfers execution to another part of the program without evaluating a condition.**

Examples

- goto
- break
- return

Goto Statement

A statement that **causes an unconditional jump to another labeled statement in the program.**

Example

```
goto label;
```


Spaghetti Code

A program structure characterized by **complex and tangled control flow caused by excessive use of goto statements.**

Problems:

- Hard to read
- Hard to debug
- Hard to maintain

Guarded Command

A control structure consisting of a **Boolean expression** (guard) and a **statement that executes when the guard is true**.

Example

- $B \rightarrow S$
- $B = \text{guard}$
 $S = \text{statement}$

Guard

A Boolean expression that determines whether the associated statement can be executed.

Guarded Selection

Example structure

```
if  
  B1  $\rightarrow$  S1  
[] B2  $\rightarrow$  S2  
fi
```

If multiple guards are true $\rightarrow$ one is chosen
nondeterministically

Guarded Loop

Example

do

$B1 \rightarrow S1$

 [] $B2 \rightarrow S2$

od

The loop continues while **at least one guard remains true**.

Modern "Guard Clauses" (Early Return)

Without Guard Clause: With Guard Clause:

```
function process(user) {  
  if (user != null) {  
    if (user.isLoggedIn) {  
      // Main Logic  
      return "Welcome";  
    }  
  }  
  return "Not Logged In";  
}
```

```
function process(user) {  
  // Guard Clauses  
  if (user == null) return "Not Logged In";  
  if (!user.isLoggedIn) return "Not Logged In";  
  
  // Main Logic  
  return "Welcome";  
}
```

Nondeterministic Selection

A selection mechanism in which **multiple guards may be true and the language allows any one of them to be chosen for execution.**